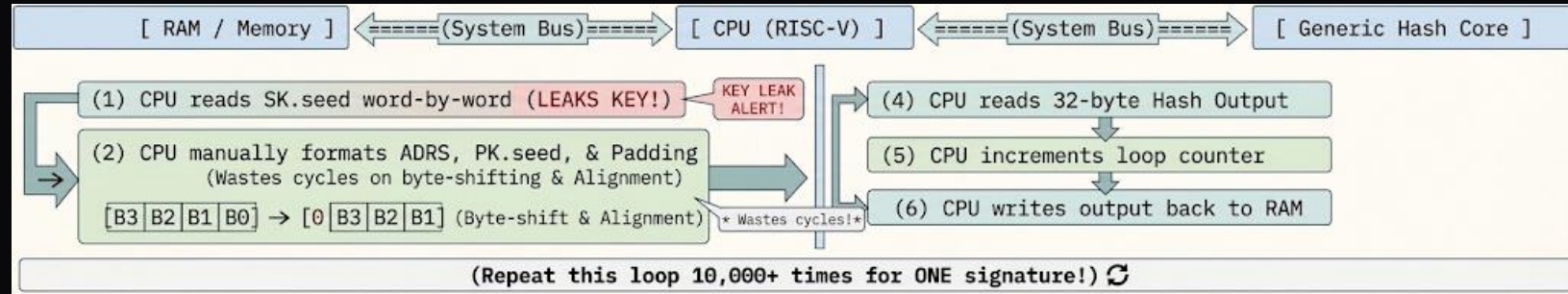


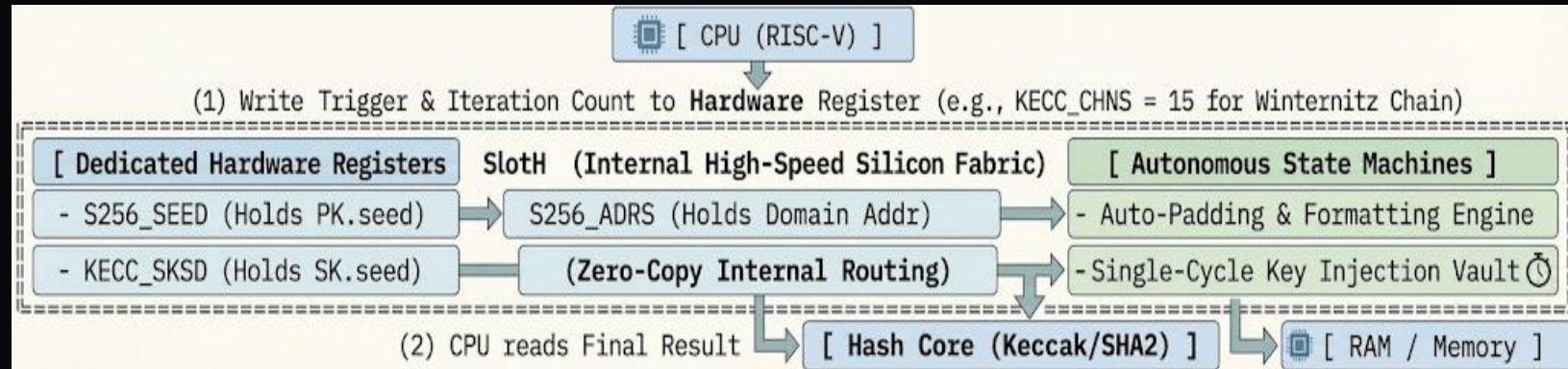
Quanta Correspondent Accelerating SLH-DSA by Two Orders of Magnitude with a Single Hash Unit

Investigating Side-Channel Attack (SCA)
Resistance and Threshold
Implementation (TI) in Post-Quantum
Hash-Based Signature: SLH-DSA.

By Adiyat Abubakirov, Tina Carter, and
Justin Williams



SLoth



The Impact Dashboard (Summary)

Metric	Standard Generic Accelerator	SLoth (Our Focus)	Why it Matters (Real-World Impact)
Speedup (vs. CPU)	~10x Faster	~300x Faster	Makes Post-Quantum Secure Boot instantaneous.
Verification Time (128s)	~4.9 Million Cycles	179,603 Cycles	Faster than legacy ECDSA! Proves PQC is viable for RoT.
Firmware ROM Size	~50+ kB (Hardcoded macros per parameter)	16.4 kB (Shared binary for all 12 sets)	Saves massive, expensive Mask ROM area in silicon.
Side-Channel (SCA)	CATASTROPHIC (Leaks in 1,000 traces)	SECURE (Single-cycle load + TI Masking)	Prevents physical attackers from stealing the master key.

Visual 3: The "Under the Hood" Cycle Comparison

What happens during ONE Winternitz Hash Iteration?

Phase of Operation	Standard CPU + Generic Hash	SLoth Co-Design
1. Key/Seed Loading	~8-16 cycles (Word-by-word from RAM)	1 cycle (Direct register injection)
2. Message Formatting	~50-100 cycles (CPU byte-shifting, ADRS padding)	0 cycles (Handled by silicon multiplexers)
3. Hash Computation	24 cycles (Keccak Permutation)	24 cycles (Keccak Permutation)
4. Loop Control & Writeback	~20-40 cycles (CPU reads, increments, writes to RAM)	1-2 cycles (Hardware state machine auto-loops)
TOTAL OVERHEAD	~100+ cycles of "wasted" CPU work	~2 cycles of hardware overhead
Hash Unit Utilization	< 20% (Starved for data)	~100% (Fed continuously)

Intellectual Merit

Problem:

- New standard for quantum-resistant digital signatures (SLH-DSA - Stateless Hash-Based Digital Signature Standard) is slow on standard computers.
- Requires millions of repetitive, heavy hash calculations (Winternitz chains) & complex message formatting.

Why it matters:

- Quantum computers are approaching and standard chips' security is unable to protect from quantum computers.

Contributions:

- Authors build SLoTH, a custom hardware accelerator that completely automates the algorithm's repetitive "grunt work" (like message padding and chaining).
- **100x to 300x speedup** compared to standard methods.
- ISSUE: Standard software secretly leaks its master key via power consumption.
- Successfully engineered a hardware shield (Threshold Implementation) to block the leak.

Impact

The paper sets up a great move in the right direction to maintain the **integrity** and **security** of firmware as we enter an age of quantum computing. **Secure boot** is a critical process to laptops, phones, and computers of all kinds, and firmware attacks on them could be devastating. The proposed ideas encourage further **collaboration between hardware and software** design when creating cryptographic capabilities. Additionally, the Hardware Accelerator built/talked about in the paper (SLoTH), is **open source**, so anyone can collaborate, build, and improve on its functions.

Our Progress

Background Research:

RoT for SoC; existing computer implementations of signature verification (with/without hash).

Coding Project Outline:

1. Core PRF Implementation

- SLH-DSA PRF (using SHAKE256) and the Winternitz chain iteration logic in Python.
- Verify against NIST Known Answer Tests (KATs) to ensure our cryptographic math is 100% compliant with FIPS 205

2. Leakage Modelling Engine

- The simulation engine that models power consumption
- **Hamming Weight** of the intermediate variables (which directly ties to the Abstract Algebra and GF(2) XOR operations we studied in **Lectures 3 & 4**).
- Model both the "leaky" CPU mode and the "protected" SLoTH mode.

3. TVLA Statistical Framework

- Use Welch's t-test to compare fixed and random key trace sets and plot to show t-value spikes as 'critical value' leaks

We Are Here